

Algorithms for Data Science

Lecture #6: Dynamic Programming

Mateusz Buczynski

University of Warsaw

15.04.2026

Thanks to dr Krzysztof Fleszar for the base material.

Dynamic Programming: The Core Idea

When DP is useful

Build the answer to a hard problem from answers to smaller subproblems.

Two signals

- Overlapping subproblems
- Optimal substructure

Main benefit

Do not solve the same subproblem again and again.

Typical DP Workflow

- 1 Define a **state**: what subproblem does one table entry mean?
- 2 Write a **recurrence**: how does this state depend on smaller states?
- 3 Specify **base cases**.
- 4 Choose an evaluation order or use memoization.
- 5 Read off the final answer, and sometimes reconstruct the solution.

Teaching rule

If the state is vague, the DP usually goes wrong.

Warm-up Example: Fibonacci Numbers

Definition

$$F_1 = 1, \quad F_2 = 1, \quad F_n = F_{n-1} + F_{n-2} \text{ for } n \geq 3$$

First values

1, 1, 2, 3, 5, 8, 13, ...

Task

Given n , compute F_n .

Naive Recursion

Direct translation of the definition

$$\text{fib}(n) = \begin{cases} 1, & n \leq 2, \\ \text{fib}(n-1) + \text{fib}(n-2), & n > 2. \end{cases}$$

Why it is attractive

The code is short and the logic matches the recurrence exactly.

Problem

The same values such as $\text{fib}(5)$ and $\text{fib}(4)$ are recomputed many times.

Fibonacci via Naive Recursion

Pseudocode

```
function fib(n):  
    if n <= 2:  
        return 1  
  
    return fib(n - 1) + fib(n - 2)
```

Cost

Simple, but exponential because the same subproblems repeat.

Why Naive Recursion Is Slow

Time recurrence

$$T(n) = T(n-1) + T(n-2) + \Theta(1)$$

Consequence

The recursion tree grows exponentially:

$$T(n) = \Theta(\varphi^n)$$

where $\varphi \approx 1.618$.

Key lesson

Short code is not the same as efficient code.

Memoization: Top-Down DP

Idea

Store each computed value the first time it appears, then reuse it.

Memoization pattern

- if n is already in memo, return it
- otherwise compute it once
- save it in memo

Effect

Each Fibonacci number is computed at most once.

Memoized Fibonacci Algorithm

Pseudocode

```
memo = empty dictionary
function fib(n):
    if n in memo:
        return memo[n]
    if n <= 2:
        memo[n] = 1
    else:
        memo[n] = fib(n - 1) + fib(n - 2)
    return memo[n]
```

Cost

Each state $1, 2, \dots, n$ is computed once, so the running time is linear.

Memoized Fibonacci: Complexity

State space

The subproblems are $1, 2, \dots, n$, so there are only n distinct states.

Average-case view with dictionary lookups

$$\text{Time} = \Theta(n), \quad \text{Space} = \Theta(n)$$

Practical view

Much faster than exponential recursion, but still uses recursion depth and cache memory.

Bottom-Up DP: Tabulation

Alternative viewpoint

Instead of asking for F_n and recursively going down, fill the answers in increasing order.

Table order

$$F_1, F_2, F_3, \dots, F_n$$

with

$$F_i = F_{i-1} + F_{i-2}.$$

Main advantage

No recursion overhead.

Bottom-Up Fibonacci Algorithm

Pseudocode

```
function fib(n):  
    if n <= 2:  
        return 1  
    F[1] = 1  
    F[2] = 1  
    for i = 3 to n:  
        F[i] = F[i - 1] + F[i - 2]  
    return F[n]
```

Cost

Iterative evaluation gives $\Theta(n)$ time with no recursion stack.

Top-Down vs Bottom-Up

Top-down

- Start from the final question
- Compute only visited states
- Natural when recurrence comes first

Bottom-up

- Start from base cases
- Fill the table in a valid order
- Often simpler and faster in practice

Decision rule

If there is a clear order on the states, bottom-up is often the cleaner implementation.

Can We Use Less Space?

Observation

To compute F_i , we only need the two previous values.

Space optimization

Keep only

$$\text{prev2} = F_{i-2}, \quad \text{prev1} = F_{i-1}.$$

Then compute

$$\text{curr} = \text{prev1} + \text{prev2}.$$

Result

$$\text{Time} = \Theta(n), \quad \text{Space} = \Theta(1)$$

A Useful Warning: Bit Complexity

What we counted so far

We treated one arithmetic operation as $\Theta(1)$.

But Fibonacci numbers grow

The number of bits in F_n is $\Theta(n)$, so adding big integers is not truly constant-time.

Takeaway

Algorithm analysis depends on the cost model. For basic DP design, the RAM model is fine; for exact arithmetic, bit complexity can matter.

What Fibonacci Teaches About DP

- 1 Identify repeated subproblems.
- 2 Store the answers.
- 3 Reuse them instead of branching exponentially.

But

Fibonacci is too easy. Real DP becomes interesting when the state has more than one dimension.

Memoization vs. Bottom-Up DP

Often interchangeable in principle.

Recursion with Memoization

- recursive
- top-down
- stores answers on demand
- may pay recursion overhead (slower)
- limited by recursion depth (risk)
- often easier to write (simpler)

Useful when the recurrence is natural
and not all states are needed.

Dynamic Programming

- iterative
- bottom-up
- fills a table in dependency order
- usually uses arrays (faster)
- no recursion stack (safe)
- evaluation order can be less obvious (harder)

Useful when the table structure is clear
and many states must be computed
anyway.

0/1 Knapsack: Problem Statement

Input

Items $1, \dots, n$, each with weight w_i and value v_i , and capacity M .

Goal

Choose a subset of items of total weight at most M with maximum total value.

Why this is a classic DP problem

Every item creates a binary decision: take it or skip it.

Optimization Formulation

Decision variables

$$x_i = \begin{cases} 1, & \text{if item } i \text{ is taken,} \\ 0, & \text{otherwise.} \end{cases}$$

Objective and constraint

$$\max \sum_{i=1}^n v_i x_i \quad \text{s.t.} \quad \sum_{i=1}^n w_i x_i \leq M$$

Why Simple Greedy Rules Fail

A tempting idea

Take the item with the best value or best value-to-weight ratio first.

Problem

For 0/1 knapsack, a locally good choice can block a much better combination later.

Teaching point

Greedy works for some optimization problems, but not for all of them.

Greedy Counterexample

Instance

$$M = 100$$

$$(v_1, w_1) = (2, 100), (v_2, w_2) = (1, 1), \dots, (v_{100}, w_{100}) = (1, 1)$$

Greedy by density

Take item 1 first, then no remaining item fits.

Greedy value = 2

Better choice

Take items 2 to 100:

Optimum = 99

Greedy Counterexample #2

Instance

$$M = 100$$

$$(v_1, w_1) = (52, 51), (v_2, w_2) = (50, 50), (v_3, w_3) = (50, 50)$$

Greedy by density

Take item 1 first, then no remaining item fits.

Greedy value = 52

Better choice

Take items 2 and 3:

Optimum = 100

Backtracking State

State definition

$\text{pack}(m, k) = \text{best value using capacity } m \text{ and items } 1, \dots, k$

Two choices at item k

- skip item k
- take item k , if $w_k \leq m$

Knapsack Recurrence

Recursive definition

$$\text{pack}(m, k) = \begin{cases} 0, & k = 0, \\ \text{pack}(m, k - 1), & w_k > m, \\ \max(\text{pack}(m, k - 1), v_k + \text{pack}(m - w_k, k - 1)), & w_k \leq m. \end{cases}$$

Naive complexity

$$\Theta(2^n)$$

because the recursion branches on many subsets.

Knapsack via Naive Recursion

Pseudocode

```
function pack(m, k):  
    if k == 0:  
        return 0  
    if w[k] > m:  
        return pack(m, k - 1)  
    return max(pack(m, k - 1),  
               v[k] + pack(m - w[k], k - 1))
```

Cost

The recursion explores many subsets, so the running time is exponential.

Memoization for Knapsack

State space

The state is a pair (m, k) , not just one number.

How many states?

$$(M + 1)(n + 1)$$

possible pairs.

Effect

Each state is solved at most once.

Knapsack via Recursion with Memoization

Complexity

At most $(M + 1)(n + 1)$ states, so time and space are both $\Theta(Mn)$.

Pseudocode

```
memo = new dictionary
function pack(m, k):
    if (m, k) not in memo:
        if k == 0:
            memo[(m, k)] = 0
        else if w[k] > m:
            memo[(m, k)] = pack(m, k - 1)
        else:
            memo[(m, k)] = max(pack(m, k - 1),
                               v[k] + pack(m - w[k], k - 1))
    return memo[(m, k)]
return pack(M, n)
```

Bottom-Up Table for Knapsack

DP table

Create $DP[0..n][0..M]$, where

$DP[k][m]$ = best value using items $1, \dots, k$ and capacity m .

Base row

$$DP[0][m] = 0 \quad \text{for all } m$$

Bottom-Up Knapsack Algorithm

Pseudocode

```
for m = 0 to M:
    DP[0][m] = 0
for k = 1 to n:
    for m = 0 to M:
        if w[k] > m:
            DP[k][m] = DP[k - 1][m]
        else:
            DP[k][m] = max(DP[k - 1][m],
                           v[k] + DP[k - 1][m - w[k]])
return DP[n][M]
```

Cost

The table has $(n + 1)(M + 1)$ entries, so time and space are both $\Theta(nM)$.

Knapsack Transition

If item k does not fit

$$DP[k][m] = DP[k-1][m]$$

If item k fits

$$DP[k][m] = \max(DP[k-1][m], v_k + DP[k-1][m - w_k])$$

Interpretation

Either the optimal solution for this state uses item k , or it does not.

Knapsack Table-Filling Order

A valid order

Fill rows for

$$k = 1, 2, \dots, n$$

and inside each row fill capacities

$$m = 0, 1, \dots, M.$$

Why this works

Every state only depends on the previous row.

Knapsack Complexity

Time

$$\Theta(nM)$$

Space

$$\Theta(nM)$$

for the full table.

Important caveat

This is polynomial in n and M , but not polynomial in the bit-length of the input when M is large.

Recovering the Chosen Items

The DP table gives the value

$$DP[n][M]$$

but we often want the actual chosen subset too.

Traceback idea

Start from (n, M) and compare

$$DP[k][m] \text{ with } DP[k-1][m].$$

If they are equal, item k was skipped. Otherwise it was taken.

Knapsack Space Optimization

Observation

Each row depends only on the previous row.

Optimization

Use only one array $DP[0..M]$, but update capacities in decreasing order.

Why decreasing order matters

It prevents one item from being used multiple times in the same iteration.

What Knapsack Adds to the DP Toolbox

- multidimensional states
- decision-based recurrence
- table reconstruction
- space optimization by reusing rows

Pattern

Many DP problems are really about making one local decision per state.

Comparing Two Sequences

Motivating question

How similar are two strings, two code versions, or two DNA fragments?

One natural notion

Look for a long sequence of symbols that appears in both inputs in the same order.

Longest Common Subsequence (LCS)

Definition

A **subsequence** may skip symbols, but must preserve order.

Example

For

$a = \text{ABCB DAB}, \quad b = \text{BDCABA}$

one common subsequence is

BCBA.

Task

Find the maximum possible length.

LCS State

State definition

$L(i, j)$ = length of the LCS of prefixes $a[1..i]$ and $b[1..j]$

Why prefixes?

They make the recurrence local: each decision only looks at the last characters.

LCS Recurrence

Case 1: last characters match

If $a_i = b_j$, then

$$L(i, j) = L(i - 1, j - 1) + 1.$$

Case 2: last characters differ

If $a_i \neq b_j$, then

$$L(i, j) = \max(L(i - 1, j), L(i, j - 1)).$$

LCS Base Cases

If one prefix is empty

$$L(i, 0) = 0, \quad L(0, j) = 0$$

Interpretation

An empty string has no non-empty common subsequence with anything.

LCS via Naive Recursion

Pseudocode

```
function L(i, j):  
    if i == 0 or j == 0:  
        return 0  
    if a[i] == b[j]:  
        return L(i - 1, j - 1) + 1  
    return max(L(i - 1, j), L(i, j - 1))
```

Cost

Without caching, many prefix pairs (i, j) are recomputed.

Memoization for LCS

Pseudocode

```
memo = empty dictionary
function L(i, j):
    if (i, j) in memo:
        return memo[(i, j)]
    if i == 0 or j == 0:
        memo[(i, j)] = 0
    else if a[i] == b[j]:
        memo[(i, j)] = L(i - 1, j - 1) + 1
    else:
        memo[(i, j)] = max(L(i - 1, j), L(i, j - 1))
    return memo[(i, j)]
return L(n, m)
```

Cost

There are only $(n + 1)(m + 1)$ states, so memoization makes the algorithm $\Theta(nm)$.

LCS Table and Complexity

Table size

$$(n + 1)(m + 1)$$

for strings of lengths n and m .

Time

$$\Theta(nm)$$

Space

$$\Theta(nm)$$

for the full table.

Bottom-Up LCS Algorithm

Pseudocode

```
for i = 0 to n:  
    DP[i][0] = 0  
for j = 0 to m:  
    DP[0][j] = 0  
for i = 1 to n:  
    for j = 1 to m:  
        if a[i] == b[j]:  
            DP[i][j] = DP[i - 1][j - 1] + 1  
        else:  
            DP[i][j] = max(DP[i - 1][j], DP[i][j - 1])  
return DP[n][m]
```

Cost

Bottom-up evaluation visits each prefix pair once.

Recovering the Actual LCS

Traceback rule

Start from (n, m) :

- if $a_i = b_j$, that character belongs to the answer
- otherwise move to the larger of $L(i-1, j)$ and $L(i, j-1)$

Result

The table can give both the length and the sequence itself.

Why LCS Is Useful

- file comparison and version control
- plagiarism detection signals
- computational biology
- sequence alignment preprocessing

Teaching point

DP is not only about numbers; it is also about structured comparisons.

Levenshtein Distance (Edit Distance)

Question

How many single-character edits are needed to transform one string into another?

Allowed operations

- insert one character
- delete one character
- replace one character

Example

kitten → sitting has edit distance 3.

Why Edit Distance Matters

- spell checking
- fuzzy matching in databases
- OCR and speech-recognition cleanup
- bioinformatics and approximate sequence matching

Difference from LCS

LCS measures the longest preserved order; edit distance measures the minimum transformation cost.

Edit Distance State

State definition

$D(i, j)$ = minimum edit distance between prefixes $a[1..i]$ and $b[1..j]$

Again

Prefixes create smaller subproblems with a clear table structure.

Edit Distance Base Cases

If one prefix is empty

$$D(i, 0) = i, \quad D(0, j) = j$$

Interpretation

To turn a length- i string into an empty string, delete i characters. To go the other way, insert j characters.

Edit Distance Recurrence

If the last characters already match

If $a_i = b_j$, then

$$D(i, j) = D(i - 1, j - 1).$$

Otherwise

$$D(i, j) = 1 + \min (D(i - 1, j), D(i, j - 1), D(i - 1, j - 1))$$

for delete, insert, replace.

Levenshtein via Naive Recursion

Pseudocode

```
function D(i, j):  
    if i == 0:  
        return j  
    if j == 0:  
        return i  
    if a[i] == b[j]:  
        return D(i - 1, j - 1)  
    return 1 + min(D(i - 1, j),  
                   D(i, j - 1),  
                   D(i - 1, j - 1))
```

Cost

The branching is large, so naive recursion quickly becomes impractical.

Memoization for Levenshtein Distance

Pseudocode

```
memo = empty dictionary
function D(i, j):
    if (i, j) in memo:
        return memo[(i, j)]
    if i == 0:
        memo[(i, j)] = j
    else if j == 0:
        memo[(i, j)] = i
    else if a[i] == b[j]:
        memo[(i, j)] = D(i - 1, j - 1)
    else:
        memo[(i, j)] = 1 + min(D(i - 1, j),
                               D(i, j - 1),
                               D(i - 1, j - 1))
    return memo[(i, j)]
return D(n, m)
```

Cost

Memoization reduces the work to the $(n + 1)(m + 1)$ distinct prefix pairs.

Reading the Three Operations from the Table

Delete

Drop a_i :

$$D(i-1, j)$$

Insert

Insert b_j :

$$D(i, j-1)$$

Replace

Replace a_i by b_j :

$$D(i-1, j-1)$$

One extra cost

Each edit costs 1.

Edit Distance Complexity

Time

$$\Theta(nm)$$

Space

$$\Theta(nm)$$

for the full table.

Space optimization

If we only need the distance value, not the actual sequence of edits, two rows are enough.

Bottom-Up Levenshtein Algorithm

Pseudocode

```
for i = 0 to n:
    DP[i][0] = i
for j = 0 to m:
    DP[0][j] = j
for i = 1 to n:
    for j = 1 to m:
        if a[i] == b[j]:
            DP[i][j] = DP[i - 1][j - 1]
        else:
            DP[i][j] = 1 + min(DP[i - 1][j],
                               DP[i][j - 1],
                               DP[i - 1][j - 1])

return DP[n][m]
```

Cost

The table has one entry per prefix pair, so the running time is $\Theta(nm)$.

LCS vs Edit Distance

LCS

- maximize agreement
- recurrence uses max
- output is a common subsequence

Edit distance

- minimize transformation cost
- recurrence uses min
- output is a number or edit script

A Common DP Design Pattern

String and sequence DP often uses

- prefixes as states
- a 2D table
- a local decision based on the last symbols

Examples

LCS, edit distance, sequence alignment, dynamic time warping.

Memoization vs Tabulation vs Space Optimization

Memoization

Natural from the recurrence.

Tabulation

Clear evaluation order.

Space optimization

Store only what future states still need.

Rule of thumb

First get the state and recurrence right. Only then optimize the implementation.

DP Checklist for New Problems

- 1 What does one subproblem mean?
- 2 Which smaller subproblems does it depend on?
- 3 What are the base cases?
- 4 In what order can we fill the states?
- 5 Do we need only the value, or also the actual solution?

What to Remember

- DP turns exponential branching into polynomial work by reusing states.
- Fibonacci shows the core idea.
- Knapsack shows multidimensional state design.
- LCS and edit distance show how DP compares sequences.
- Good DP starts from a precise state definition.

Next Time

Coming next

We move from algorithm design to data structures for efficient lookup, updates, and large-scale approximate queries.